Звіт до комп'ютерного практикуму №1

Нейронні мережі прямого поширення

ПІБ: Балабан Ілля Олегович
Група: ІК-61мп
Номер варіанта: 2
Датасет і тип задачі: Concrete Compressive Strength (Регресія)
Два дослідження за варіантом: Активація (ReLU, tanh, ELU); Dropout (0, 0.2, 0.5)

Мета роботи: ознайомитися з принципами побудови, навчання та використання нейронних мереж прямого поширення для розв'язання задач класифікації й регресії; набути практичних навичок підготовки даних, створення та навчання моделей засобами Keras; дослідити вплив структури мережі, функцій активації, алгоритмів навчання та їхніх гіперпараметрів на якість роботи моделі.

Завдання: підготувати дані; створити й оцінити базову модель Keras; окремо провести два контрольовані дослідження відповідно до варіанта; обрати найкращі значення параметрів, об'єднати їх у покращеній моделі та порівняти результати моделей.

Засоби виконання практикуму: [середовище виконання; Python; Keras; додаткові бібліотеки та їхні версії].

1. Набір даних і попередня обробка

1.1. Характеристика датасету. Указати джерело, кількість об'єктів і ознак, цільову змінну, кількість класів для класифікації або діапазон цільових значень для регресії, наявність пропущених значень та інші суттєві особливості.

1.2. Підготовка даних. Описати кодування категоріальних ознак і цільової змінної, масштабування, обробку пропусків та формування навчальної, валідаційної й тестової вибірок у співвідношенні 70% / 15% / 15%. Пояснити, як уникнуто витоку даних.

Вибірка	Кількість об'єктів	Частка
Навчальна	722	70%
Валідаційна	154	15%
Тестова	155	15%

2. Базова модель

2.1. Архітектура та налаштування. Навести схему і фрагмент коду створення моделі Input → Dense(32, ReLU) → Dense(16, ReLU) → Output. Обґрунтувати вихідний шар, функцію втрат і метрики відповідно до типу задачі. Зафіксувати Adam, learning_rate=0.001, batch_size=32 і максимальну кількість 100 епох.

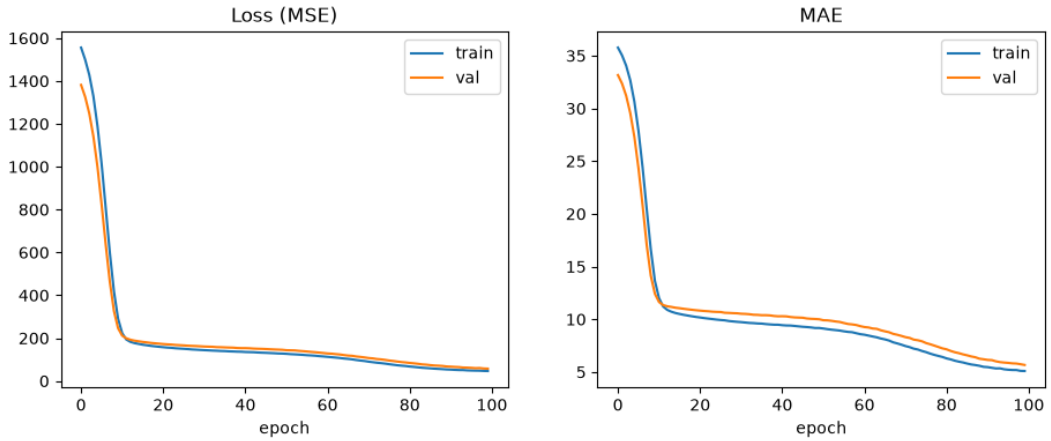
2.2. Навчання. Навести фрагмент коду навчання із зазначеним callback та вказати фактичну кількість епох і час навчання.

```
early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True,
)
```

```
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    batch_size=32,
    epochs=100,
    callbacks=[early_stop],
)
```

2.3. Результати базової моделі. Подати графіки train/validation loss і основної метрики, числові результати та коротко оцінити наявність недонавчання або перенавчання.

Показник	Результат
Кількість епох до зупинки	100
Час навчання	9sec
Validation loss / основна метрика	val_loss: 63.3452
Початкові тестові метрики	MAE = 5.34781736455634, RMSE = 6.52871943301937, R2 = 0.830356661685242



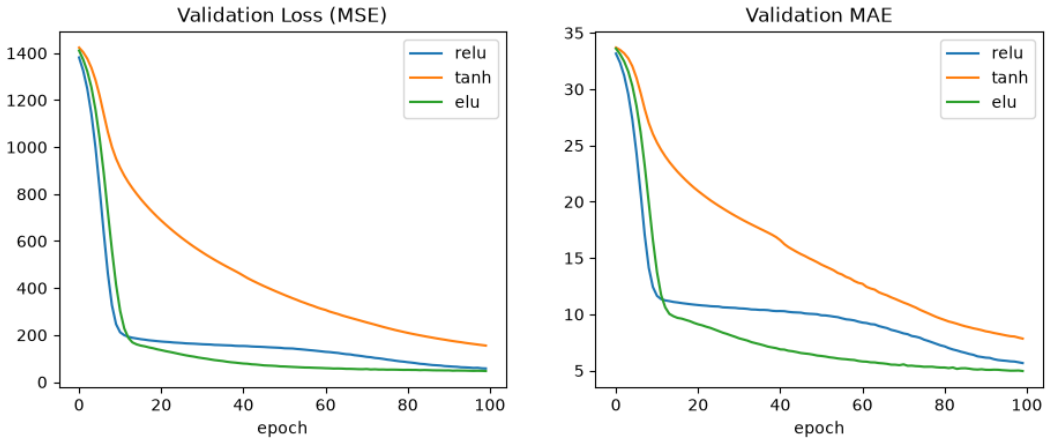
Модель навчалась протягом усіх 100 епох і не зупинялась, що свідчить про покращення достатнє аби не активувати EarlyStopping, що вказує на недовнавчання.

3. Контрольоване дослідження 1

3.1. Мета й умови. Указати параметр, який змінюється, досліджувані значення та очікуваний вплив. Підтвердити, що всі інші налаштування відповідають базовій моделі. Для вибору найкращого значення використовувати валідаційну, а не тестову вибірку.

Значення параметра	Епох до зупинки	Час	Validation loss / метрика
ReLU	100	8sec	val_loss ≈ 56.07
tanh	100	9sec	val_loss ≈ 155.00
ELU	100	9sec	val_loss ≈ 46.78

3.2. Аналіз і висновок. Додати порівняльний графік або діаграму, пояснити виявлену залежність і назвати найкраще значення параметра для покращеної моделі.



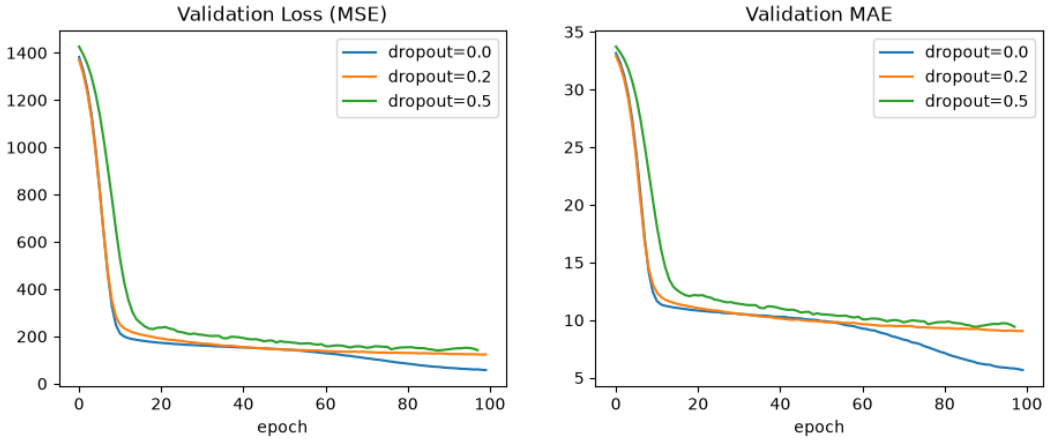
Tanh показав себе найгірше і є найповільнішим, ReLU та ELU досить близько, проте ELU сходиться швидше та має найнижчі значення val_loss та val_mae.

4. Контрольоване дослідження 2

4.1. Мета й умови. Указати другий параметр, його досліджувані значення та очікуваний вплив. Проводити експеримент окремо від дослідження 1, змінюючи лише цей параметр відносно базової моделі.

Значення параметра	Епох до зупинки	Час	Validation loss / метрика
Dropout 0	100	8sec	val_loss ≈ 56.07, val_MAE ≈ 5.63
Dropout 0.2	100	8sec	val_loss ≈ 123.68, val_MAE ≈ 9.05
Dropout 0.5	98	8sec	val_loss ≈ 141.77, val_MAE ≈ 9.42

4.2. Аналіз і висновок. Додати порівняльний графік або діаграму, пояснити виявлену залежність і назвати найкраще значення другого параметра.



Dropout показав себе досить погано, про що свідчать результати на графіку, з Dropout = 0 модель показує найкращі результати, коли з 0.2 та 0.5 досить швидко виходить на плато. Результати свідчать що Dropout тільки заважає навчанню.

5. Покращена модель

5.1. Формування моделі. Об’єднати найкращі значення параметрів із двох досліджень. Навести структуру, ключові налаштування та фрагмент коду фінальної моделі. Інші умови навчання, зокрема EarlyStopping, залишити такими самими, якщо зміни не передбачені варіантом.

```
model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(32, activation="elu"),
    keras.layers.Dense(16, activation="elu"),
    keras.layers.Dense(1, activation="linear"),
])

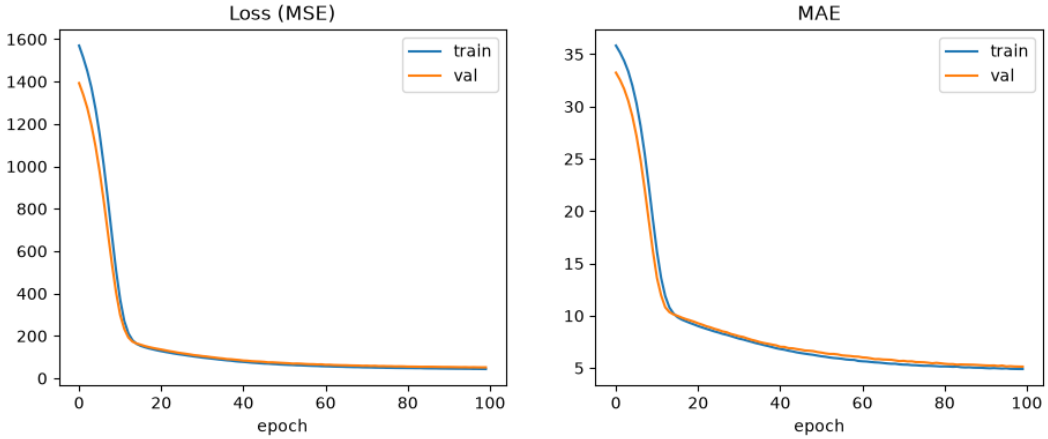
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="mse",
    metrics=["mae"],
)

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True,
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    batch_size=32,
    epochs=100,
    callbacks=[early_stop],
)
```

Згідно попередніх досліджень найкраща комбінація це активація ELU та Dropout = 0.

5.2. Результати. Навести графіки навчання, фактичну кількість епох, час і результати на тестовій вибірці. Для класифікації подати accuracy, macro F1 та confusion matrix; для регресії — MAE, RMSE, R^2 і, за можливості, графік прогнозованих та фактичних значень.



ELU, Dropout = 0 - 8 секунд, 100 епох

6. Порівняння моделей

Модель	Ключові налаштування	Тестові метрики	Епохи / час
Базова	Dense(32, ReLU) → Dense(16, ReLU) → Dense(1, linear) Без Dropout Adam, learning_rate = 0.001, batch_size = 32	MAE = 5.3478 RMSE = 6.5287 R^2 = 0.8304	100 епох / 9 с
Покращена	Dense(32, ELU) → Dense(16, ELU) → Dense(1, linear) Dropout = 0 Adam, learning_rate = 0.001, batch_size = 32	MAE = 5.1633 RMSE = 6.5449 R^2 = 0.8295	100 епох / 8 с

Порівняльний аналіз.

Якість. На тестовій вибірці MAE зменшився з 5.3478 до 5.1633 (−3.5 %), тоді як RMSE (6.5287 → 6.5449) і R^2 (0.8304 → 0.8295) практично не змінилися; на валідаційній вибірці val_loss знизився з 56.07 до 51.78 (−7.6 %), а val_MAE — з 5.63 до 5.13 (−8.9 %).

Час і швидкість збіжності. Обидві моделі пройшли 100 епох без спрацювання EarlyStopping; час навчання однаковий у межах похибки (≈ 9 с проти ≈ 8 с). ELU збігається швидше: поріг val_loss ≤ 100 досягнуто на 34-й епосі проти 66-ї у ReLU (val_loss ≤ 60 — на 69-й проти 96-ї), хоча перші ~10 епох вона навчається дещо повільніше.

Очікуваність результату та узагальнення. Оскільки найкращий Dropout = 0 (як у базовій моделі), об'єднання звелося до заміни ReLU на ELU, і очікуваний ефект дослідження 1 (швидша збіжність, нижчі val_loss та val_MAE) підтвердився. Розрив між train і validation зменшився (за loss — з 11.04 до 7.93, за MAE — з 0.61 до 0.22), але на тесті виграш майже непомітний, а кожну модель навчали один раз без фіксованого seed (у двох запусках базової моделі test R^2 становив ≈ 0.69 та ≈ 0.83), тому покращення узагальнювальної здатності слід вважати помірною тенденцією, а не підтвердженим фактом.

7. Висновки

Загальні висновки. Стисло сформулювати результати роботи: вплив кожного з двох досліджених параметрів, найкращу конфігурацію, переваги або обмеження покращеної моделі, наявність недонавчання чи перенавчання та можливі напрями подальшого покращення.

Досліджено нейронну мережу прямого поширення (Input → Dense(32) → Dense(16) → Output) для регресії міцності бетону на стиск (Concrete Compressive Strength); базова модель із ReLU дала на тесті MAE ≈ 5.35 МПа, RMSE ≈ 6.53 МПа, R^2 ≈ 0.83. Активація: найкраща ELU (val_loss ≈ 46.78 проти 56.07 у ReLU та 155.00 у tanh — tanh збігається значно повільніше). Dropout: найкращим є 0, значення 0.2 і 0.5 погіршили val_loss до 123.68 і 141.77, бо в невеликій мережі без помітного перенавчання dropout лише зменшує її ємність. Найкраща конфігурація — ELU, Dropout = 0 (Adam, learning_rate = 0.001, batch_size = 32, EarlyStopping з patience = 10).

Покращена модель збігається швидше (val_loss ≤ 100 за 34 епохи проти 66) і має менший розрив train/validation, проте на тесті виграш незначний (MAE 5.16 проти 5.35 МПа, RMSE і R^2 без змін); обмеження — по одному запуску без фіксованого seed та окремий, а не спільний аналіз параметрів. Обидві моделі пройшли всі 100 епох, а val_loss ще знижувався, що вказує на легке недонавчання; суттєвого перенавчання немає. Напрями покращення: збільшити кількість епох або learning_rate, розширити мережу, дослідити batch_size і оптимізатор, усереднювати результати за кількома seed, за потреби застосувати L2-регуляризацію замість dropout.

Додаток А. Програмний код

Вміст додатка. Навести повний програмний код або посилання на доступний викладачеві notebook. У основній частині звіту залишити лише фрагменти коду, потрібні для пояснення підготовки даних, архітектури та навчання моделей.

Рекомендований обсяг основної частини: 4–6 сторінок без урахування додатка

https://colab.research.google.com/drive/1t0o4oKps2rvOyDKLfSfD7h3-6lTEwIVq?usp=sharing